1 Lesson 16-10-26

1.1 Sequential logic

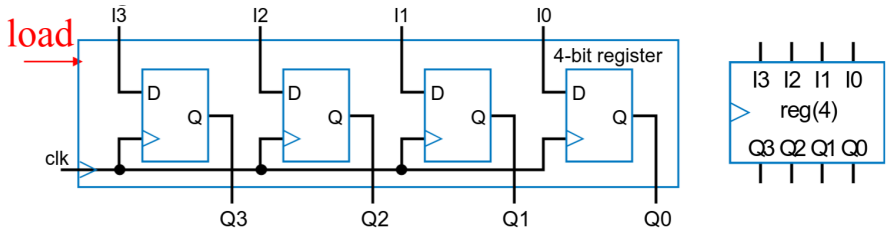
1.2 Sequential datapath components

Let's look at sequential components applied to the datapath of a circuit.

1.2.1 Registers

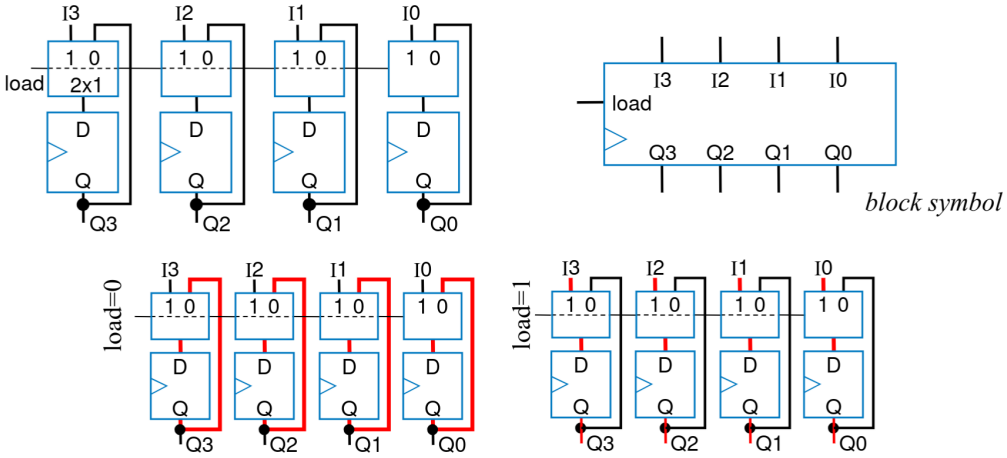
We have introduced **registers**. In general, an N-bit register stores N bits, where N is also known as the width.

A register is loaded every cycle. It is useful for implementing FSMs, as it stores encoded state (the **STAR**, *STAtus Register*).



1.2.2 Parallel load register

We can have a register *conserve* its value for more than one clock cycle by implementing a **parallel load** topology. The idea is to add a mux before the register's data input, which can select between an external (load) output and the register's own output.

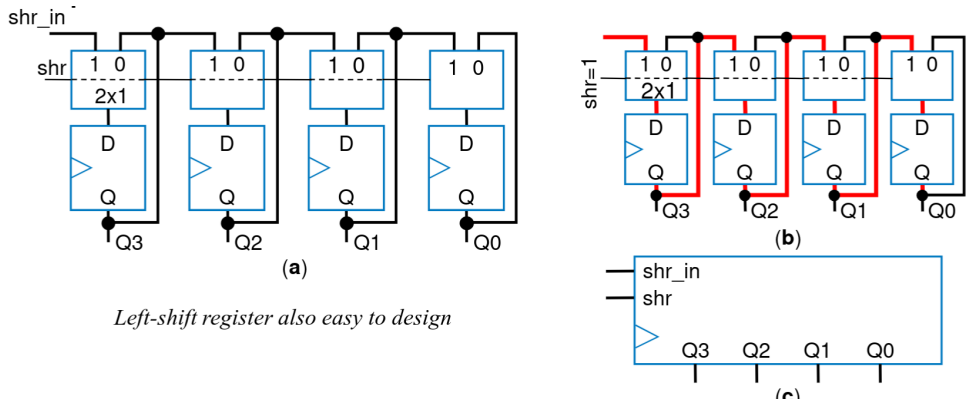


1.2.3 Shift register

An extension of the parallel load register is basically any registers which allows either to conserve or perform some operation on its contents at each clock cycle. One such popoular device is the **shift register**.

The idea of a shift register is to move each bit one position right (left) in the register width, with the rightmost (leftmost) bit is "dropped", that is 0 shifts into into leftmost (rightmost) bit.

We can easily achieve this by chaining flip-flops:



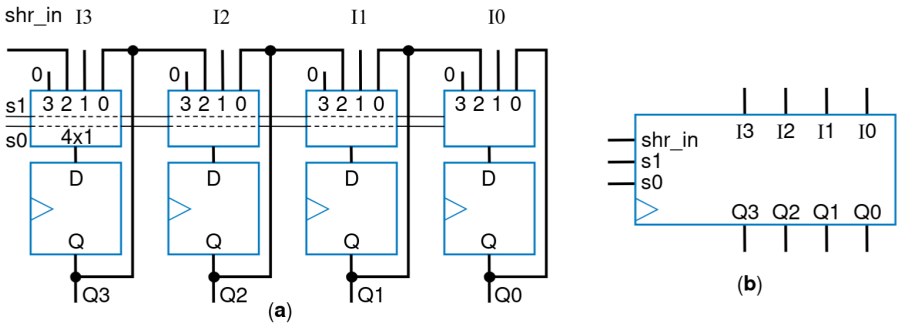
Left-shift register also easy to design

1.2.4 Multifunction register

At this point we would like to generalize registers to implement various functions, that is implement a **multifunction register**.

A multifunction register has encoded control inputs instead of decoded control inputs (one for each function): Many registers have multiple functions, such as load, shift, clear (load all 0s), and are also able to retain present value (as in a parallel load register).

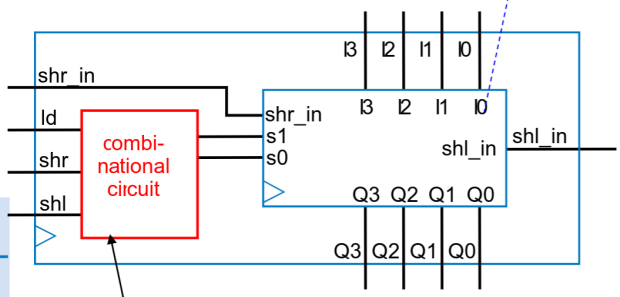
These are easily designed using muxes: we ust connect each mux input to achieve the desired function.



Note that usually we prefer to encode functions in binary values (with each number indexing a function). If we want single hot-encoded bits signaling a certain operation, we're better off using a combinatorial network to get back to a binary encoding:

ld	shr	shl	Operation
0	0	0	Maintain value
0	0	1	Shift left
0	1	X	Shift right
1	X	X	Parallel load

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	Shift left



Truth table for combinational circuit

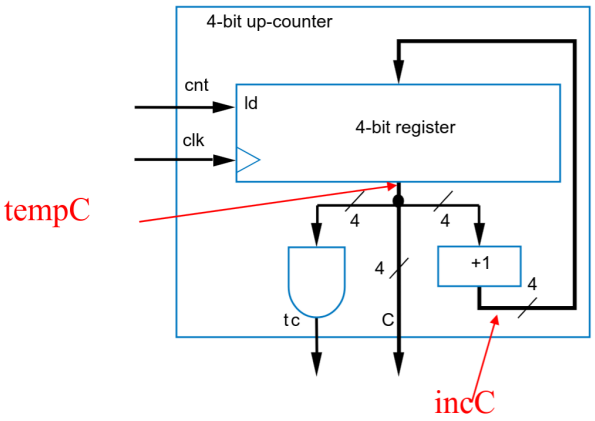
Inputs ld	shr	shl	Outputs s1	s0	Note Operation
0	0	0	0	0	Maintain value
0	0	1	1	1	Shift left
0	1	0	1	0	Shift right
0	1	1	1	0	Shift right
1	0	0	0	1	Parallel load
1	0	1	0	1	Parallel load
1	1	0	0	1	Parallel load
1	1	1	0	1	Parallel load

$s1 = ld'shr'shl + ld'shrshl' + ld'shrshl$
$= ld'shl + ld'shr$

$s0 = ld'shr'shl + ld$

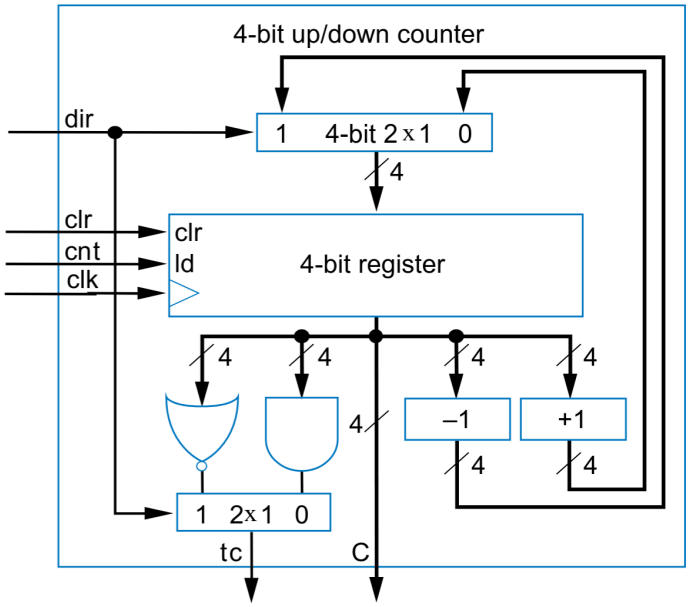
1.3 Counters and Timers

A **counter** is a sequential circuit based on an N-bit register that can increment (add 1) to its own value on each clock cycle.



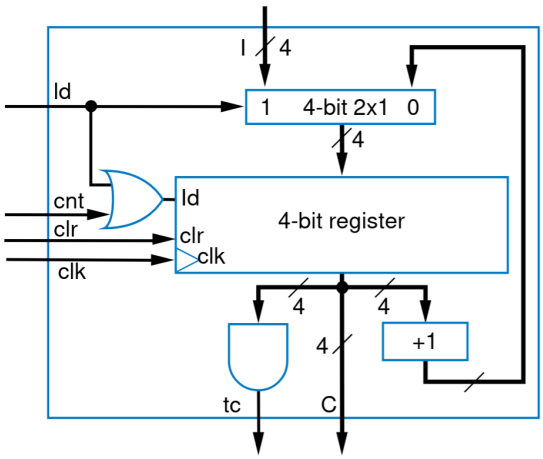
We see that the up counter design is simple: basically a multifunction register with a clear function and a increment function (extra optimizations can be hidden away in the incrementer network).

A complication we're interested in, on the other hand, is having *up/down* counters.



Let's now look into including a *load* feature in our counter.

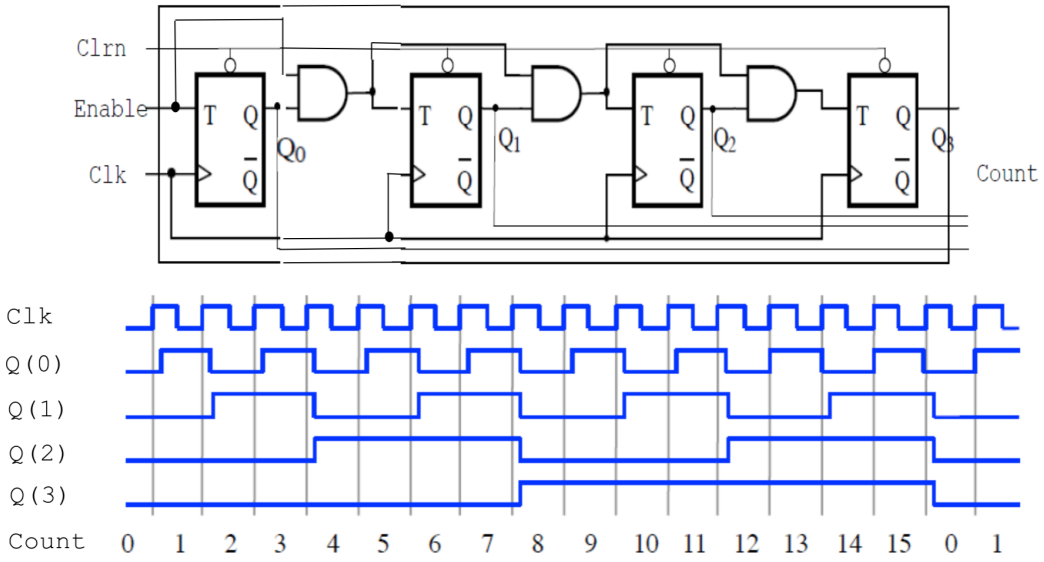
For instance, an up-counter that can be loaded with an external value is designed using a 2 to 1 mux. The `ld` input selects the incremented value or the external value. The still load the internal register when loading external value or when counting (via an OR gate).



Note that in this design `ld` has priority over `clr` (because of the conditioning combinatorial network at the inputs).

1.3.1 Synchronous implementation

We see one possible implementation for an N-bit counter with enable and clear.



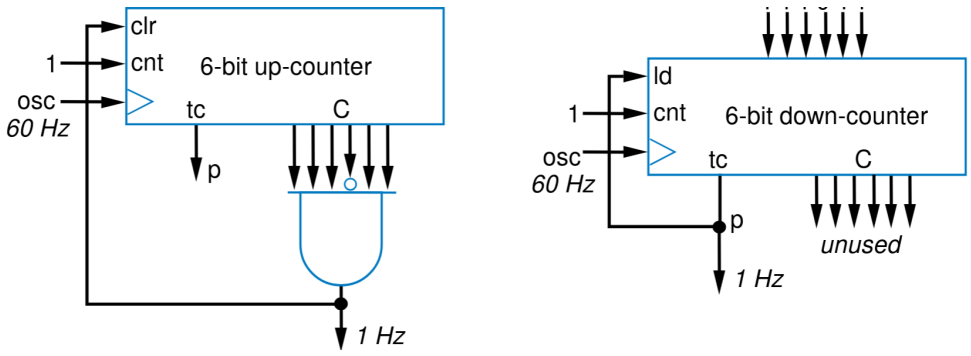
1.3.2 Modulo counters

An interesting class of counters is used to **divide clock** cycles. What we can do is basically connect the `tc` output of a down counter w/ load to its `ld` input. This means that the counter will load anything that's fed to it, the moment it stops counting (thus counting to values k, different than a power of 2).

Let's now see what we mean when we say dividing a clock. uppose have 512 Hz oscillator, but want a 2 Hz pulse. We want to divide this fast clock by 256. Noting that $2^8 = 256$, thus an 8-bit counter counts 256 cycles, we can design a divider using an 8-bit up-counter, using the `tc` output as a pulse signal.

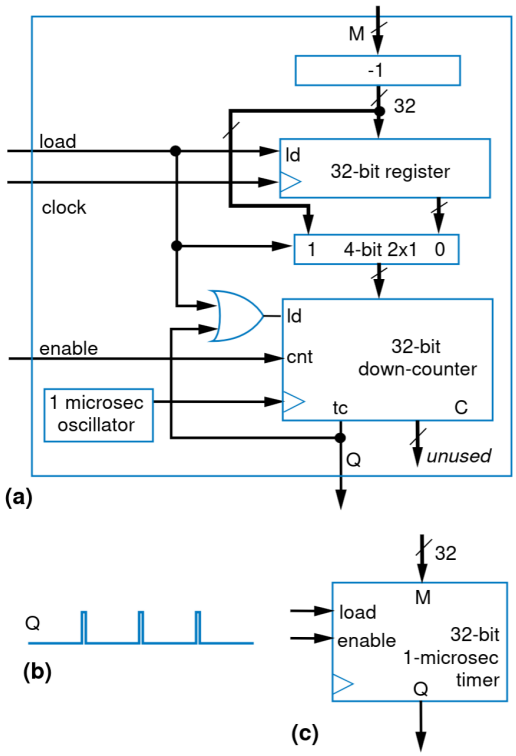
At this point the above design for a k-modulo coutner makes sense to divide signals by dividers not a power of 2. Note that an up counter approach is also possible, but more expensive, by matching with an AND gate at the output.

Let's give a final figure, showing up and down counter approaches to dividing a 60 Hz clock to a 1 Hz pulsing signal.



1.3.3 Timers

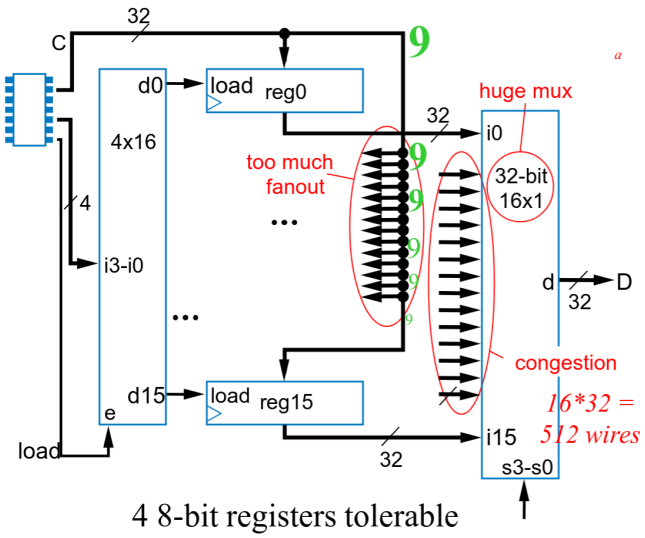
Lastly, let's see the **timer**. It is a device that pulses the output at a user-specified timer interval when enabled. The interval is clearly specified as a multiple of the base time unit. This can be designed using an oscillator, register, and down-counter:



1.4 Register files

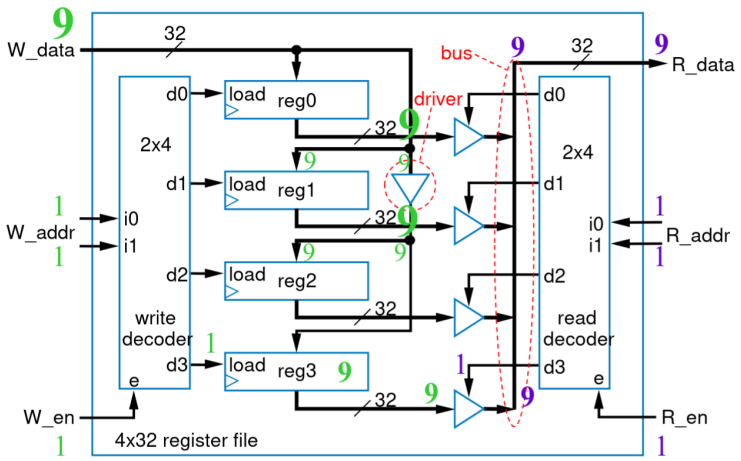
An $M \times N$ register file is a set of M registers each N bits wide. It should ideally allow for one-at-a-time, synchronous writes, and one-at-a-time (or multiple) asynchronous reads.

The simple approach, involving simple registers with load enables and a big output multiplexer, doesn't really work as it requires a huge output multiplexer (with extremely high signal degradation).



4 8-bit registers tolerable

This, a good internal design uses drivers and buses to ensure signal strength.



Internal design of 4x32 RF